Table of Contents

1 Introduction

The music industry has undergone drastic changes in the last two decades, both due to the rapid digitalization of content distribution and the emergence of accessible streaming platforms. The transition from physical media to digital libraries and, later, to cloud-based streaming has altered how people view, consume, and interact with music. While in the early 2000s the main channels for music discovery were friend groups and magazines, platforms such as Spotify, Apple Music, and YouTube have revolutionized music consumption, enabling listeners to explore growing catalogs through personalized recommendation systems. These systems, originally rooted in collaborative filtering and content-based filtering ([8], [50]), shaped the modern music industry and listener experience.

These methods relied upon the assumption that a user's listening history could inform recommendation to users with similar interests (collaborative filtering) and that content with similar features derived from metadata and audio characteristics could be recommended as alternatives (content-based filtering).

Such algorithms rapidly became central to music discovery, and redefined the economic model of the industry itself, incentivizing record labels and artists to optimize songs for algorithmic visibility and user retention rather than traditional artistic or album-oriented coherence ([20], [38], [42]). This shift has enhanced the importance of catchiness, brevity, and immediate accessibility features, that align with modern consumption patterns and algorithmic promotion ([43], [26]).

Notwithstanding the downsides that such technology imposed upon the artistic world, its emergence drastically improved the user experience and allowed a larger proportion of the population to discover their favorite artists, contributing to expand the reach of music itself.

More recently, advances in natural language processing and machine learning improved upon existing recommendation systems and led to the emergence of emotion-aware music recommendation; such algorithms aim to capture not only user preferences but also affective states and contextual cues ([56]).

Moreover, the advent of transformer and generative technologies vastly improved the set of tools available for these tasks, with chatbots continuing to rise in popularity as a consequence ([28], [2]). In fact, transformer-based embeddings have proven to enhance the quality of retrieval ([30], [22]), while generative models address long-standing limitations

of older recommendation systems, such as the cold-start problem (recommending to new users is challenging as there is no data on their listening history), and allow for better contextualization of the user's preferences and affective state ([28], [2]).

The combination of the aforementioned developments in NLP led to the spread of Retrieval-Augmented Generation (RAG), with vast applications both in the literature and the industry; the latter, however, shows larger practical interest in such technologies, due to the market demand for AI assistants and the need to ground generated outputs on company information and private databases ([23], [3]; [52]). In fact, the literature shows gaps in experimental deployment of RAG architectures (albeit leading in theoretical development) ([7]), and to our knowledge applications in the music sector are still to be explored.

In this thesis, I build a novel dataset of song-level information such as lyrics, annotations and audio features for 220k songs, sourcing data from Spotify (song title, artist, metadata), Genius (lyrics, annotations) and ReccoBeats (audio features). We contribute to the literature by exploiting said dataset to build a RAG recommender system for music suggestion, with specific focus on emotional understanding of the user's prompts. To my knowledge, there is no evidence of a similar large-scale implementations.

I test retrieval and generation separately, finding promising results for both, with strong emotional alignment between the emotion conveyed by the retrieved songs and the user's emotion in writing the prompt, and strong contextualization abilities from the chosen LLMs.

I believe that this approach to recommendation could improve upon the previously mentioned flaws of algorithmic suggestions, as the final user has more control over the candidates they are presented with, contributing to a more personalized and interactive experience.

2 Related work

This section reviews foundational and contemporary work relevant to this thesis, spanning three main areas. First, it outlines advances in Natural Language Processing (NLP) that enabled machines to extract and represent semantic meaning from text. Next, it examines developments in music recommendation, focusing on collaborative, content-based, and hybrid approaches that historically address user modeling and item representation. Finally, it explores Retrieval-Augmented Generation (RAG), a recent development that enhances language models with external knowledge retrieval, highlighting its potential to improve factual grounding in recommendation contexts.

2.1 Natural Language Processing

Natural Language Processing (NLP) is a field that aims at transforming written text into rich data, which can be an extraordinary asset for a variety of machine learning tasks. As the volume of textual data grew exponentially during the early 2000s and 2010s, due to the rapid digitalization of information, researchers suddenly had the means to expand upon existing research (older neural networks) and develop technologies that would shape the current world.

While early research revolved around sparse methods to represent large bodies of text, (such as TF-IDF which relies on word frequency), the focus quickly shifted toward neural representation learning, with distributional embeddings like word2vec ([36]) and GloVe ([41]) begin able to capture the semantic meaning of words with simple neural networks. While these models capture semantic similarity through static embeddings, they could not account for contextual variation in word meaning; recurrent neural networks, especially LSTMs ([27]) and GRUs ([13]), introduced a way to capture the context of a sentence, albeit limited in doing so.

The introduction of attention mechanisms ([4]) eased these limitations by allowing models to selectively focus on relevant input content. This laid the ground for the emergence of the transformer architecture, which removed recurrence and enabled parallel, long-context computation, which was first introduced in "Attention Is All You Need" ([4]). On top of this architecture, training procedures on massive corpora of text yielded powerful general-purpose language models such as BERT ([16]) and GPT ([45]), whose performance has

been rapidly improving ever since. Alongside the performance gains, the general-purpose nature of LLMs allows them to be adapted to specific tasks through lightweight fine-tuning, eliminating the need to retrain large models from scratch and enabling efficient transfer learning ([55]).

LLMs experienced massive investment and interest (both from the literature and the industry), consolidating as the state of the art for many older NLP tasks such as classification and semantic similarity, while simultaneously making text generation (likely the most challenging task in the field) a feasible reality.

2.2 Music Recommendation

Modern music platforms expose users to catalog sizes that induce decision fatigue; recommender systems are the primary mechanism for discovery and retention ([8]). Classical frameworks fall into collaborative filtering (learning from user–item interactions) and content-based methods (learning from item descriptors and audio). Each provides complementary strengths but also well-documented weaknesses ([32], [1], [21]). Later implementations reveal hybrid models that incorporate the best of both worlds, with multimodal signals and graph structures improving older architectures.

Focusing on collaborative filtering (CF), as described by Knees and Schedl ([31]) and Celma ([8]), this strategy operates on the assumption that users with similar listening histories will enjoy similar tracks. This domain-independent approach made CF widely applicable since it relies solely on user–item interaction data rather than musical content. CF proved particularly effective in identifying community-driven patterns, improving accuracy for active users with extensive interaction histories.

Such a method, however, suffers from inherent weaknesses. The most significant being data sparsity and the cold-start problem. In fact, many songs do not show enough data to be recommended and similarly, recommendations for new users are challenging due to the lack of a listening history ([31]). Moreover, CF tends to reinforce popularity bias, thus recommending well-known tracks and excluding niche or emerging artists ([33], [54]). This tends to hinder diversity, leading to repetitive recommendations that limit user discovery. The approach proposed in this thesis manages to overcome such limitation, as each track is recommended based on semantic similarity between the lyrics and the user request, hence, the model is agnostic as to the popularity of the song.

Content-Based Filtering, on the other hand, approaches the recommendation problem from the opposite perspective, analyzing the intrinsic properties of the music itself. According to Fu et al. ([21]) and Celma ([8]), CBF uses metadata such as genre, artist, and album information, or low-level acoustic features including Mel-Frequency Cepstral Coefficients and spectral descriptors so that new or less popular songs can be recommended as well.

Metadata-based systems, however, suffer from incomplete or inaccurate tagging, while purely signal-based models often fail to capture higher-level user preferences ([51], [15]). These shortcomings benefited from recent improvements in standard machine learning techniques, with Cunha ([14]) showing how these methods can complement metadata with song similarity scores that help to capture semantic meaning. Furthermore, deep learning advancements have improved CBF through the use of Convolutional Neural Networks (CNNs) and transformer-based architectures to extract relevant deep audio embeddings ([19]), which better capture songs and their similarities, thus facilitating the recommendation task.

Lastly, hybrid approaches emerge as the clear winner, due to their ability to mitigate popularity bias, which affects user satisfaction and engagement. By balancing exposure between popular and niche content, hybrid systems provide a fairer recommendation landscape, ensuring that users discover new music while still aligning with their tastes ([49], [37]). In particular, methods based on Graph Neural Networks (GNNs) excel across many metrics and are proven to better promote diversity and personalization ([5])

2.3 RAG

LLM-based systems have been shown to be powerful but delicate: they hallucinate facts, lag on recent information, and struggle to incorporate structured domain knowledge (catalogs, knowledge graphs, playlists) at inference time. RAG (Retrieval-Augmented Generation) was born to address these gaps by fetching authoritative, up-to-date evidence during inference and conditioning the generator on it, improving faithfulness and controllability without retraining the model.

In practice, these models are hybrid architectures that couple a retriever with a LLM-based generator, allowing the model to pull relevant external passages from a database at inference time and condition text generation on that data.

In order to enable the retrieval operation, documents in the knowledge base need to be split into smaller chunks, and be encoded into dense vector representations. These vectors are then indexed into efficient data structures that facilitate retrieval (through clustering algorithms and nearest-neighbor search), and stored in a vector database.

Retrievers can then compute dense vector representations of queries and search over a large index using Dense Passage Retrieval ([30]), where the query vector is compared to the documents' vectors to fetch the top-K most similar passages (using vector similarity metrics such as the L2-norm or cosine similarity). Modern vector-store frameworks such as FAISS partition the embedding space via k-means clustering into centroids, then narrow the search to the nearest centroids before exploring their member vectors, hence reducing search cost (and improving scalability) while retaining high accuracy ([18]).

Retrieved passages are then fused with the prompt within a transformer-based generator (LLM), enabling factual grounding and reducing hallucinations ([35], [46]).

REALM ([24]) is one of the first implementations that, although not referring to music-based applications, shows how training retrieval and language modeling jointly, improves alignment between search and generation. Such an architecture has been proven to be effective in multimodal settings as well ([11], [57]).

Concerning the applications of RAG to the recommendation task, recent work in movie recommendation explicitly motivates the use of this architecture, showing that external retrieval mitigates knowledge-gaps-and reduces hallucinations in LLM-Rec pipelines, with knowledge graph based retrieval further enhancing performance ([25]).

While one of the main concerns with these systems is the computational cost at inference time, models such as OptiRAG-rec ([58]) show that adjustments to the model's architecture (i.e., early exit mechanisms) can increase efficiency without affecting the accuracy of the retriever.

While implementations specific to music recommendations are yet to be explored, some work with similar scope has been conducted. Although distinct from classical RAG architectures, Text2Tracks ([40]) grounds LLM generation in the item catalog, but accomplishes this internally by storing track representations in model parameters and directly generating item IDs instead of retrieving evidence.

Similarly, TALKPLAY ([17]) aligns with the broader RAG objective of grounding generation in the item catalog, in this case by expanding the LLM's vocabulary to include

multimodal item tokens, enabling the model to recommend directly from structured music content rather than hallucinating unseen items. TALKPLAY reformulates conversational music recommendation as a next-token prediction task, introducing a multimodal music tokenizer that encodes audio, lyrics, metadata, semantic tags, and playlist co-occurrence into discrete music tokens the LLM can generate.

Lastly, MusT-RAG ([34]) exploits a RAG architecture to perform text-based music question answering. The authors adapt general-purpose LLMs to music domain knowledge by retrieving authoritative textual context from MusWikiDB, a curated vector database designed for music-focused factual QA.

Collectively, existing work demonstrates the importance of grounding LLM behavior in structured musical knowledge. Current approaches, however, either focus on generative catalog access, limiting controllability and filtering (Text2Tracks, TALKPLAY), or apply RAG only to knowledge-centric tasks without modeling preference signals (MusT-RAG). As a result, none explicitly integrates lyrical semantics and metadata retrieval to support context-aware track suggestion. **This thesis addresses this gap by leveraging lyrics-based dense retrieval and annotation-grounded re-ranking to deliver semantically aligned music recommendations.**

3 Data

The following section describes the data collection strategy adopted to construct a comprehensive dataset suitable for implementing the proposed recommendation framework. The process integrates three complementary data providers: Spotify for song metadata, Genius for lyrical and annotated content, and ReccoBeats for audio-level descriptors, ensuring that a broad range of musical features are included. Subsequently, the final dataset is described in detail.

3.1 Data collection

The motivation for the construction of a novel dataset stems from three main issues:

1. **Lacking features**: Similar datasets lack the variety of features required for this task; many provide lyrics without audio features or Genius annotations, and often miss the relevant identifiers to retrieve them efficiently from other sources.

2. **Limited scope**: Popular datasets used for similar tasks lack an up-to-date music library and neglect older tracks.

3. **Sample size**: Apart from a few exceptions which nevertheless fall under the previous two categories, the prevailing majority of existing datasets do not provide a satisfactory sample size, necessarily resulting in worse recommendations.

3.1.1 Spotify

The first building block of the data strategy exploits Spotify's API to fetch a considerable and well-formatted sample of song titles, artists and Spotify IDs, which will then be the key to retrieve information from other sources. In order to understand the queries described later in the section, it is paramount to understand the two main limitations posed by this API:

- ◇ **Rate limit**: Spotify allows developers to fetch a maximum of 1,000 songs per query
- ◇ **Relevance**: After filtering through the query, results are retrieved according to an internal relevance ranking, which is not disclosed to the public.

Attempting to amend the structural flaws of existing datasets, a custom logic is implemented. First, the United States' market is defined as the main target for queries, as historically it best represents music culture, both in terms of diversity and volume. Secondly, I set a temporal span: as an ideal objective, I attempt to fetch music from 1960 to 2025 (as it will emerge in the following sections, the data will be skewed towards recent tracks due to availability). Lastly, a macro-genre map (Table 1) is designed, assigning to every decade a coherent sample of genres to include in the query. The advantages of this approach are threefold:

1. Genre information is not included in any of the API endpoints (only as a query filter), and the information would otherwise be lost.

2. It allows for a coherent and structured output, as only the most relevant genres will be considered each year, thus reducing the number of API calls (e.g. Hip-Hop will not be fetched for the '60s, '70s and '80s)

3. It introduces diversity and granularity in the dataset, enabling the inclusion of less popular songs, given that a genre itself might be less popular than another one and block it from ever appearing in the results.

Once the setting has been defined, I iterate through each year included in the study, as well as every genre for the respective decade, trying to fetch the top 1000 songs for each iteration. In turn, this results in a hypothetical total of eighteen thousand tracks per year, or roughly 1.1 million songs overall. Such a target is unfortunately far from reality due to the limited availability for certain genres and earlier years, as well as duplicated tracks. The actual retrieved dataset includes 871,820 songs, although after de-duplicating instances that appear multiple times (due to cross-genre overlap and the presence of different versions of the same song within the same genre) the final dataset reduces to 627,546 unique songs.
Regarding the features considered for inclusion at this step, we denote the following:

- **Title**: Song title.

- **Artist**: Artist name. In case multiple artists are featured in the same track, their names are concatenated into a single string for compactness, with the first always being the main artist.

- **Spotify_id**: The unique identifier for Spotify tracks, which will prove fundamental in section 3.1.3.
- **Year**: Year the song was released.
- **Album**: Album name.
- **Release_date**: Exact release date of the song.
- **Popularity index**: Popularity index based on number of recent plays for a given song.
- **ISRC_code**: Universal identifier for a music recording, which despite not being used in this study, could be of interest for further research.

Genre	60s	70s	80s	90s	00s	10s	20s
Alternative	X	X	✓	✓	✓	✓	✓
Blues	✓	✓	✓	✓	✓	✓	✓
Bossanova	✓	✓	X	X	X	X	X
Country	✓	✓	✓	✓	✓	✓	✓
Dance	X	X	✓	✓	✓	✓	✓
Disco	X	✓	✓	X	X	X	X
Electronic	X	X	✓	✓	✓	✓	✓
Folk	✓	✓	✓	✓	✓	✓	✓
Funk	✓	✓	✓	✓	✓	✓	✓
Gospel	✓	✓	✓	X	X	X	X
Hip-hop	X	X	X	✓	✓	✓	✓
Indie	X	X	✓	✓	✓	✓	✓
Jazz	✓	✓	✓	✓	✓	✓	✓
Latin	X	X	X	✓	✓	✓	✓
Metal	X	✓	✓	✓	✓	✓	✓
Pop	✓	✓	✓	✓	✓	✓	✓
Punk	X	X	✓	✓	✓	✓	✓
R&B	✓	✓	✓	✓	✓	✓	✓
Rap	X	X	X	✓	✓	✓	✓
Reggae	X	✓	✓	✓	✓	✓	✓
Rock	✓	✓	✓	✓	✓	✓	✓
Singer-songwriter	✓	✓	✓	✓	✓	✓	✓
Ska	✓	X	X	X	X	X	X
Soul	✓	✓	✓	X	X	X	X

Table 1: List of genres and their inclusion status by decade

3.1.2 Genius

Once a suitable sample of songs has been constructed, the next step of the data strategy relies on leveraging the Genius API to gather data on song lyrics and annotations.
Genius is a user-driven platform and digital media company that lets users annotate and interpret song lyrics, literature, news and other texts. The platform was founded in 2009 as "Rap Genius", initially focusing solely on hip-hop, but quickly expanded to broader genres and types of content. On Genius, registered users can highlight lines of text and add contextual notes, images or links; these annotations can be voted on by the community and verified by the artist. Artists can also annotate their own work, making Genius both fan-based and creator-driven. The platform thus embodies the largest available database for song lyrics and semantic music discussion, justifying the choice to include it as a data source for the purposes of this study.
Concerning the data collecting procedure, I first chunked the 627k results from Spotify into smaller batches of approximately 5000 songs (occasionally adjusting manually due to rate limit quotas and the sensitivity of the API to connectivity drops). Subsequently, leveraging the query filters for song title and artist name, I fetched the HTML result page for each track. However, as was stated previously, Genius contains a broad range of different media, hence the first result is not necessarily the lyrics page for the song (i.e. it could be a relevant news article). For this reason, I fetch multiple results and iterate until a lyrics page is returned. Moreover, after testing the website and the responses, it emerged that the search engine is often imprecise and can yield songs with the same title but different artists and vice versa. That is why I implement a strict similarity threshold, through Ratcliff-Obershelp similarity ([47]), and fetch the song only if the Genius strings for the artist name and song title exceed this threshold. Furthermore, for each request, the main artist is selected as the key together with the song title, but if no result is returned and other artists are featured, the full string of artists will be passed in a second request. The rationale behind fetching the HTML code instead of the specific endpoint is purely computational: for some songs, Genius annotations are included in the page HTML and require a separate API call otherwise. This means that the HTML result allows to avoid this call entirely for a subset of the dataset and speeds up the process.
After the described procedure roughly 66% of the observations obtained in section 3.1.1 are lost, reducing the final sample size to 219,261 songs with lyrics, of which 47,139 include

annotations. In the case of lyrics, the data loss is due to the imprecise nature of the Genius API when artist name and song title are used as keys for the query; in fact, knowing the genius ID ex ante would result in minimal loss isolated to cases where the song does not have a lyrics page. On the other hand, not all songs boast annotations, as they are mainly created by the community, and thus subject to variability on this front.
Lastly, the data are merged with the Spotify data adding the following features:

- **Lyrics**: String containing the lyrics of the song, as well as a brief descriptive header for songs which provide it (see section 3.2).
- **Annotations**: A list of Genius annotation strings.
- **Genius_ID**: Not relevant for the purposes of this study but useful for further research.
- **Genius_URL**: URL for every song pointing to the lyrics webpage; included for similar reasons as the previous point.

3.1.3 Other sources: ReccoBeats

ReccoBeats is a cloud-based music intelligence platform designed to extract and provide high-level audio features for large collections of songs. This service had been one of the core endpoints of Spotify's API, until it was deprecated in April 2025, thus motivating the addition of a third data source. ReccoBeats offers an accessible API that enables developers to retrieve standardized descriptors such as acousticness, danceability, energy, instrumentalness, liveness, loudness, speechiness, tempo, and valence. These metrics quantify different musical attributes, ranging from rhythm and timbre to emotional tone. In particular, the valence feature measures the emotional positivity of a track on a scale from "sad" to "happy", making this data especially suitable for emotion-aware applications. In fact, they will contribute to amplify the context fed to the LLM, thus improving generation and interpretation.
Accessing this information is straightforward as the platform comes with a free-access API. Its database is updated regularly and optimized, providing coverage for millions of tracks across diverse genres and eras. Furthermore, fetching song-level information is possible directly through a track's Spotify ID, without needing to match on strings (as with Genius' API). The final dataset has audio information for 198,988 songs out of the

219,261 songs included (91%). A full list of audio features, together with their description as provided by ReccoBeats is reported in table 2

Table 2: Description of Audio Features Extracted from ReccoBeats

Feature	Description
`acousticness`	Confidence (0.0–1.0) that the track is acoustic. Higher values indicate more natural sounds.
`danceability`	Suitability for dancing (0.0–1.0). Higher values indicate more rhythmically engaging tracks.
`energy`	Intensity and liveliness (0.0–1.0). Higher values indicate more energetic tracks.
`instrumentalness`	Likelihood of no vocals (0.0–1.0). Values above 0.5 suggest instrumental tracks.
`liveness`	Probability of a live audience (0.0–1.0). Values above 0.8 strongly suggest a live track.
`loudness`	Average loudness in decibels (dB). Typically ranges between –60 dB and 0 dB.
`speechiness`	Presence of spoken words (0.0–1.0). Values above 0.66 indicate mostly speech.
`tempo`	Estimated tempo in beats per minute (BPM). Typically ranges between 0 and 250 BPM.
`valence`	Emotional tone (0.0–1.0). Higher values indicate a happier mood, lower values a sadder one.

3.2 Data description

The final dataset comprises 219,261 tracks, collected across three main data sources: Spotify (metadata), Genius (lyrics and annotations), and ReccoBeats (audio features). The collection process followed the multi-stage pipeline illustrated in Figure 1.

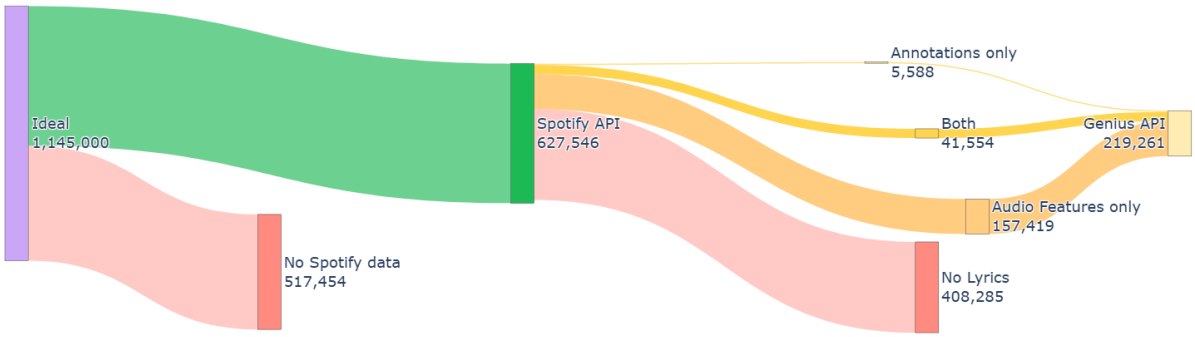

Figure 1: Overview of the sample and data sources

Regarding the temporal dimension, the dataset spans from 1960 to 2025. Figure 2 shows that the yearly distribution is skewed towards recent years, which is to be expected as the availability for older tracks is limited on Spotify's API. However, the overall representation decade-wise is satisfactory and it improves upon older data sources. This holds especially true for more recent tracks, from 2020 to 2025, which are currently not covered by any publicly available dataset with comparable feature depth and diversity.

On the other hand, the genre distribution (Figure 2) falls a bit short of expectations, failing to cover popular genres such as *hip-hop* or *reggae* in a satisfactory manner. This is due to underlying limits of the API, where songs falling in these genres might be mis-categorized or require greater granularity in the genre definition (e.g., trap, boom-bap, etc.). While this result might be disappointing, major music genres are well represented with *rock*, *electronic*, *dance*, *indie*, *jazz*, *folk*, *pop*, *r&b*, *punk*, *metal* and country all exceeding 10,000 observations, thus constituting roughly 96% of the dataset.

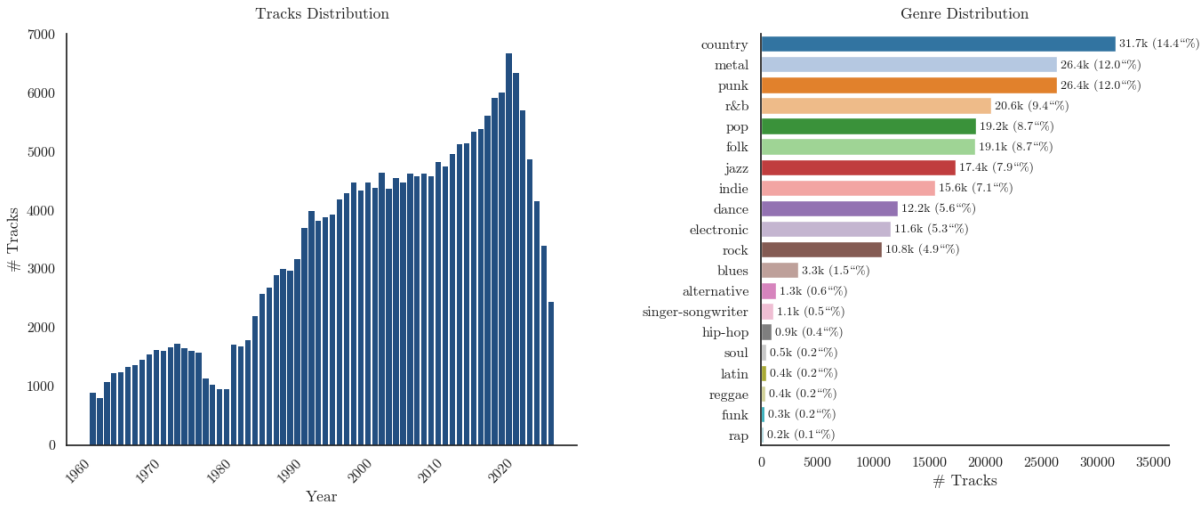

Figure 2: Distribution of tracks across time and genres

Moreover, by analyzing the audio feature distributions (Figure 3), I find that the dataset provides a comprehensive overview of the musical and acoustic diversity. Overall, the sample is dominated by studio-produced, vocal, and energetic tracks, reflecting the general characteristics of contemporary streaming catalogs. Features such as acousticness and instrumentalness are highly skewed toward low values, indicating that most songs rely on electronic or amplified instrumentation and include vocals rather than being purely instrumental. Energy values concentrate near the upper end of the scale, suggesting that high-intensity and dynamically rich tracks prevail. Danceability follows a moderately normal distribution, which corresponds to rhythmically engaging but not exclusively dance-oriented music. The tempo distribution, peaking between 110 and 130 BPM, aligns with the dominant tempo range of pop, electronic, and mainstream genres. Meanwhile, liveness and speechiness display strong left-skewness, implying that live or spoken-word recordings constitute a minor portion of the dataset. Valence, which is paramount to the context of this study, is roughly uniform with a slight tendency toward neutral and negative affect, suggesting a broad emotional spectrum. Finally, loudness peaks around -8 dB, reflecting modern mastering conventions that favor high playback volume. Together, these observations confirm that the dataset mirrors the characteristics of contemporary digital music consumption, hence providing a suitable sample for the context of this analysis.

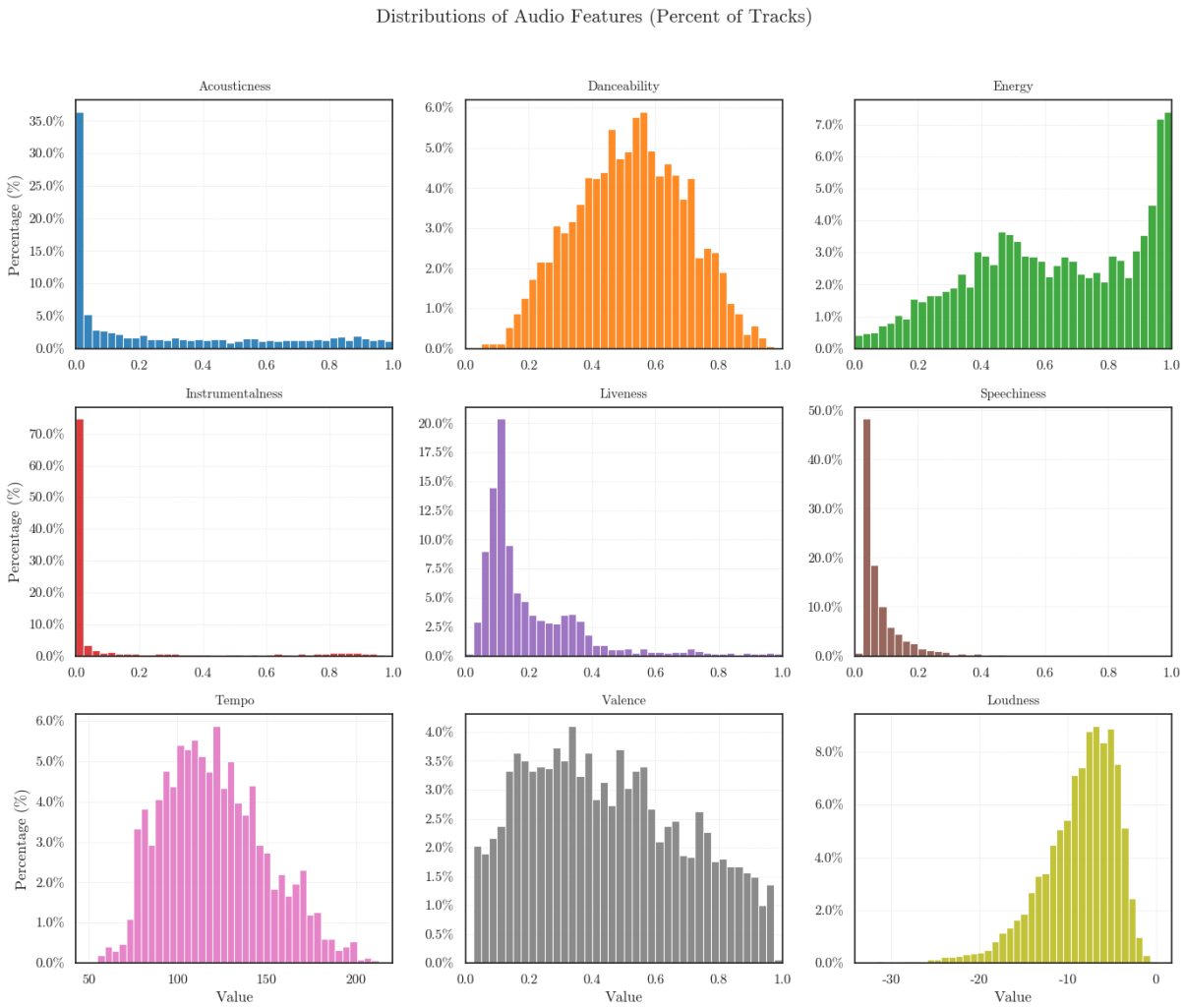

Figure 3: Audio features' distributions

Furthermore, the audio feature vector provides information that could help discern stylistic differences across genres. Figure 4 reports the results of K-Means clustering applied to dimensionally reduced audio vectors through Principal Components Analysis.

In both the 2D and the 3D projections, the clusters exhibit decent separation along their principal components, with Metal tracks (orange) forming a dense cluster toward higher PC1 values and Folk tracks occupying the opposite side of the vector space. This result aligns with the nature of the two genres, with the former being more energetic, loud and less acoustic than the latter. R&B (Green) spans a broader area between the two, as expected; in fact, this genre shares features with both clusters, as it often varies between more energetic and mellow tracks. The 3D projection confirms the presence of somewhat defined clusters for these genres also along the third principal component.

Overall, this result confirms the power of these features, and their ability to capture mean-

ingful stylistic information.

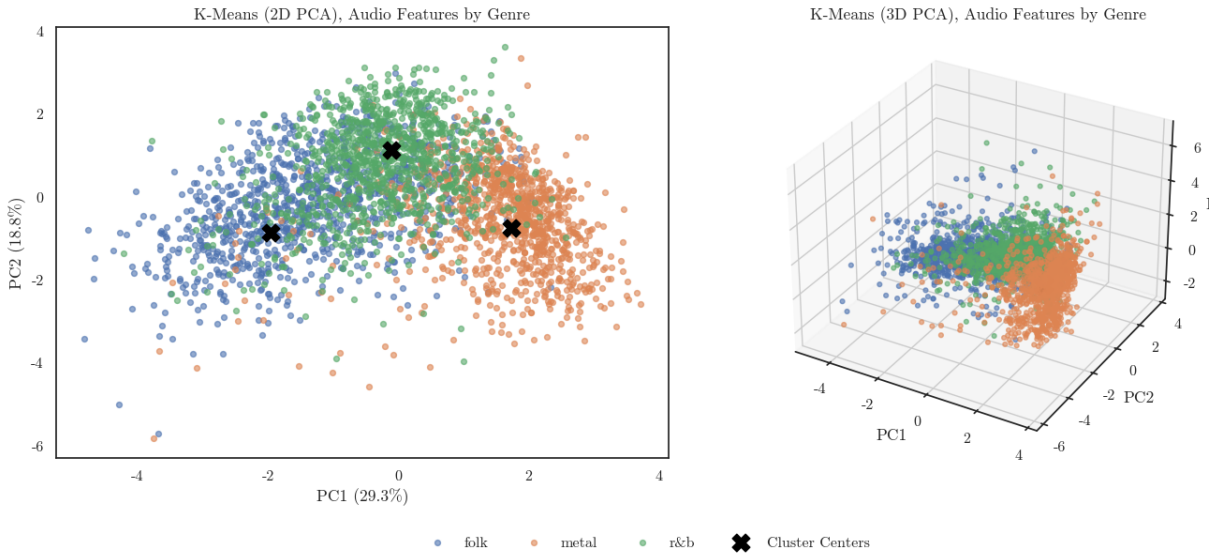

Figure 4: K-Means clustering of audio fearures vectors after PCA

4 Methodology

This section describes the methodological framework adopted to develop, implement, and evaluate the proposed system. It serves to illustrate the architectural design and experimental setup that guided the study.

4.1 RAG

This study implements a Retrieval-Augmented Generation (RAG) framework for music recommendation, designed to integrate dense semantic retrieval with generative reasoning. The system leverages the LangChain framework ([9]) to orchestrate the architecture, context formatting, and large language model (LLM) generation.

LangChain is an open-source framework (and platform) designed to simplify the development of applications that integrate LLMs (as well as many other machine learning models) and make computation pipelines more accessible. It provides a structured programming framework and Python libraries that allow developers to build their applications; this is particularly true for LLMs, due to the optimal integration with Ollama and HuggingFace. While neither the former nor the latter needs LangChain to deploy models, they do benefit from the structure it provides. In the context of this research, LangChain was used to implement the Retrieval-Augmented Generation (RAG) pipeline entirely in Python.

The architecture follows the canonical RAG design: a retriever component searches a vector database for semantically relevant items given a user query, while a generator uses this contextual information to produce natural-language playlist recommendations.

The overall workflow proceeds as follows: first, song lyrics from a corpus of 219,988 tracks are embedded into a high-dimensional vector space using a transformer-based embedding model. These embeddings are indexed in a Facebook AI Similarity Search (FAISS, [18]) vector store, allowing the system to perform a similarity search between the user prompt and the lyrics. Upon receiving a user query, the retriever returns the top-k relevant tracks, whose textual and audio-feature metadata are formatted into a predefined prompt. The prompt, together with the user's request, is then passed to a generative LLM (e.g., LLaMA-3), which outputs a playlist of five songs along with short justifications for each song choice.

4.1.1 Vector Database

Formally, following Kwon et al., 2025 ([34]), the retriever R can be defined as a function:

$$R : (q, D) \rightarrow c$$

where q denotes the embedded user query, D represents the entire collection of song lyric embeddings, and $c \subset D$ is the retrieved subset of documents that best match the query. Each document $p \in D$ is scored by the cosine similarity between the query and document embeddings:

$$\text{sim}(q, p) = \frac{E(q) \cdot E(p)}{\|E(q)\|\|E(p)\|}, \tag{1}$$

where $E(\cdot)$ denotes the embedding function computed by the encoder model. The retriever ranks all documents in D by their similarity scores and selects the top-k results, which form the contextual input for the generative model. In this study, D corresponds to the set of dense embeddings of song lyrics and associated metadata, while R is implemented through a FAISS-based retriever integrated within the LangChain framework. Since FAISS requires L2-normalized vectors, the inner product can be computed for similarity as it is equivalent to the cosine similarity.

In practice, the vector store is constructed using dense embeddings computed with *BGE-M3* ([10]), implemented via the `FlagEmbedding` library and integrated into LangChain through a custom wrapper class. The model choice stems from the need for having a model that excels at retrieval tasks, while allowing for multi-lingual data, due to the variety of idioms that characterize the US music market ([39], [10]). Each track's lyrics are thus encoded into 1024-dimensional dense vectors using the model's sentence encoder on a CUDA device and L2-normalized.

The large embedding size should help to capture semantic information from song lyrics, enabling the retrieval of emotionally aligned songs even when lexical overlap is minimal. Embedding vectors are subsequently indexed with FAISS, due to its ease of integration within the LangChain framework and competitive performance.

The experimental setup employs a flat index, which performs direct inner-product simi-

larity computations between the query and the stored embeddings and by doing so, performs exact retrieval. While FAISS allows to boost performance with large-scale Nearest-Neighbor search and vector compression algorithms such as product quantization, it comes with an accuracy trade-off, as well as increased computational cost for training; hence, since the scale of the database allows for near real-time exact-retrieval, the flat index is the preferred choice. Indexing is performed in batches of 10,000 tracks, after which sub-indexes are merged into a single global store.

Each song is then stored as a LangChain *Document* object containing the lyrics as `page_content` and a `metadata` dictionary including identifiers (`spotify_id`, `isrc`), textual metadata (`title`, `artist`, `genre`, `release_date`), and audio features (`acousticness`, `danceability`, `energy`, `valence`, etc.). Genius annotations, when available, are added to provide interpretative context for the LLM. The final database, comprising both embeddings and metadata, is serialized and stored locally.

Lastly, at inference time, retrieved documents are transformed into concise textual candidate descriptions. Each candidate is presented with title, artist, selected audio features, a truncated lyric snippet (150 characters), and Genius annotations. Figure 5 shows the candidate's structure.

4.1.2 Generation

The generation stage of the Retrieval-Augmented Generation (RAG) system constitutes the second and final step of the recommendation pipeline, in which the retrieved musical documents are transformed into personalized textual outputs for the end user. After the retriever provides the top-k (typically k=20) most semantically relevant songs from the vector database, each result is preprocessed and formatted into a structured string context containing the available supporting data for context (Figure 5). Specifically, the system concatenates for each candidate song: the title, artist, release year, genre, and audio features (e.g., valence, energy, etc.), followed by a truncated snippet of lyrics and the associated annotation text (if present).

The generation component was implemented using the LangChain framework, which provides a modular interface for constructing prompt-based reasoning chains. The pipeline integrates a *ChatPromptTemplate* (available from LangChain) defining two distinct message roles for the underlying LLM: the system prompt specifies the task, defines the

```
"""
Candidate 1:
Title: {title}
Artist: {artist}

Year : {year}
Release_date : {release_date}
Genre : {genre}
Popularity : {popularity}
Acousticness : {acousticness}
Danceability : {danceability}
Energy : {energy}
Instrumentalness : {instrumentalness}
Key : {key}
Liveness : {liveness}
Loudness : {loudness}
Mode : {mode}
Speechiness : {speechiness}
Tempo : {tempo}
Valence : {valence}

Lyrics snippet: {first 150 characters of lyrics}
Annotation snippet: {joined Genius annotations}

"""
```

Figure 5: Structure of the formatted candidate representation passed to the LLM. Each retrieved document is converted into a textual block containing basic metadata (title, artist, genre, acoustic features), a short lyric snippet, and any available annotation text.

constraints on the output and gives definitions for the audio features. The user prompt reports the user's request with a list of candidates to choose from.

The system prompt, as reported in Figure 6, explicitly frames the model as a music curator whose objective is to select and justify five optimal songs to suggest to the user from the retrieved set. Additionally, it forces a fixed output format consisting of a numbered list where each item includes a "Title - Artist" pair followed by a short justification as to why that song was suggested. The instructions further constrain the generation style to be concise, non-repetitive and most importantly, emotionally aligned with the user's query.

```
SYSTEM_PROMPT = """
You are curating a playlist for a user given their request.

You have candidate songs with audio features and short
   lyric/annotation snippets.
Pick the BEST 5 for the user and speak directly to them.

Rules:
  - Write as a friendly curator and music advisor.
  - Use the metadata to best adapt your playlist to the user
     request.
  - For each pick, include a concise 'Why this fits' explanation.
     You can reference lyrics and use annotations to explain.
  - Use also audio features to guide your choice, but DON'T ever
     mention them to the user. They are your internal information.
  - Pick songs that match the user perceived emotion, it is very
     important that you base your choice on emotion.
  - Keep each explanation to 1  2  sentences.
  - Avoid repeating the same reason across songs.
  - Output EXACTLY this format:

  1) Title - Artist
    Why this fits: ...
  2) Title - Artist
    Why this fits: ...
  3) ...
  4) ...
  5) ...

Context:

"{full audio features descriptions as per table 2}"
"""
```

Figure 6: System prompt given to the LLM

Regarding the Large Language Model itself, three distinct ones were included in the study:

- **Llama3 - 7b** ([53])
- **Mistral - 7b** ([29])
- **Qwen2.5 - 7b** ([44])

In particular, the Ollama platform allows users to run LLM-based applications locally (and fast) by providing a terminal interface and an API to pull the model weights, as well as optimization/quantization for these models. Combined with its easy integration within the LangChain framework and open-source access, Ollama was the clear choice for this study's pipeline. Concerning the model parameters (constant across models), a lower *temperature* setting (0.3) was chosen to minimize randomness, yielding more deterministic, recommendation-like responses aligned with the system's role as a "structured curator". Moreover, an extended context window was selected ($num\_ctx = 8192$) to allow the model to process large textual inputs containing multiple song candidates without truncation, ensuring that all relevant metadata and lyrical information could influence generation. Additionally, the maximum amount of output tokens was set to 512 ($num\_predict = 512$), to ensure that responses were not cut.
Lastly, the final output (after generation) is passed through LangChain's `StrOutputParser()` for readability.

4.2 Evaluation

The evaluation of the proposed Retrieval-Augmented Generation (RAG) system aims to assess its capacity to recommend music that aligns with the emotional intent expressed by the user. Two complementary evaluation components were designed: one focusing on emotional relevance of the retrieved results (Section 4.2.1) and another on the quality of the generative recommendations (Section 4.2.2).

4.2.1 Emotional relevance

In order to assess whether the retriever is able to suggest relevant music that captures the user's emotion, a multilingual emotion classifier, `MilaNLProc/xlm-emo-t` ([6]), was

employed for automatic annotation for both the user prompts and the lyrics of the retrieved songs. The classifier distinguishes four emotions: joy, sadness, fear, and anger, and outputs the highest-probability label for a given corpus.

The objective of the evaluation is to assess whether the predicted emotion for a given prompt (e.g. *anger*) is aligned with the predicted emotion of the lyrics of the songs that were retrieved following such prompt. To this end, two curated sets of 20 prompts each were designed to test the system under different linguistic conditions:

- ◇ **Emotionally explicit prompts**, which directly convey the emotion
- ◇ **Emotionally implicit prompts**, in which the emotion is implied through imagery or entities rather than lexical cues.

Prompts were constructed following the 3Q framework ([12]). This framework models how humans infer emotions that are not explicitly stated in language by guiding reasoning through three consecutive cognitive steps: *What* is being talked about, identifying a key entity, *Why* it is mentioned (it should be evocative of an emotional situation), and *How* the speaker feels about the entity (implying the underlying emotion). Following the 3Q rationale, implicit prompts were crafted to require inferential reasoning about emotional context rather than direct recognition of affective adjectives. Each implicit prompt encodes emotion through imagery, figurative language, or situational framing, forcing the retriever to rely on semantic associations captured in the embedding space rather than lexical features.

Each prompt is tagged as either implicit or explicit and embedded using the BGE-M3 model described in section 4.1.1. The vector is then fed to the retriever, which yields the top 50 most similar results, and hence their lyrics. The emotion classifier then predicts the dominant emotion for each retrieved text (truncated to 512 characters due to model requirements). The resulting dataset contains prompt–result pairs with their associated predicted emotions and retrieval ranks.

An overview of the evaluation workflow is shown in Figure 7, and the full list of prompts is reported in tables 3a and 3b.

To evaluate emotional relevance and ranking quality, several standard retrieval metrics were computed at different cut-offs (k = 5, 10, 20, 50):

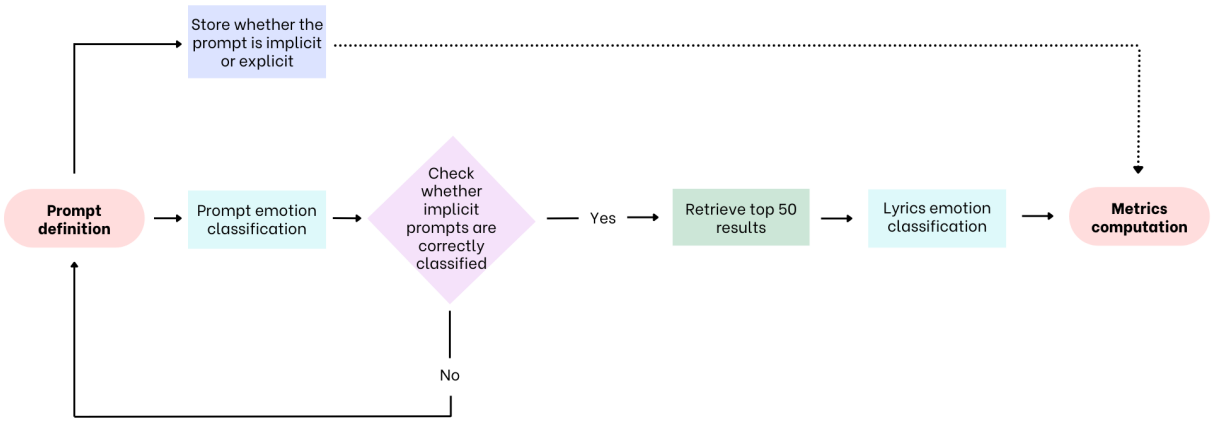

Figure 7: Overview of the evaluation pipeline for the emotional alignment

- **Precision@k**, **Recall@k**, and **F1@k** (macro-averaged): assess how many of the top-k retrieved lyrics share the same emotion as the prompt.
- **Mean Reciprocal Rank** (MRR@k): measures how early the first emotionally matching song appeared in the ranking.

$$\text{MRR@k} = \frac{1}{M} \sum_{i=1}^{M} \frac{1}{r_i} \tag{2}$$

where M is the total number of prompts (queries), and r_i is the rank position of the first retrieved lyric whose predicted emotion matches that of the i-th prompt ($r_i = \infty$ if no relevant lyric is retrieved within the top-k).

- **Normalized Discounted Cumulative Gain** (nDCG@k): evaluates the overall ranking quality by rewarding emotionally relevant lyrics appearing higher in the ranking. It is defined as:

$$\text{nDCG@k} = \frac{1}{M} \sum_{i=1}^{M} \frac{\text{DCG}_i@k}{\text{IDCG}_i@k} \tag{3}$$

with

$$\text{DCG}_i@k = \sum_{j=1}^{k} \frac{rel_{ij}}{\log_2(j+1)} \qquad \text{and} \qquad \text{IDCG}_i@k = \sum_{j=1}^{|R_i|} \frac{1}{\log_2(j+1)} \tag{4}$$

where $rel_{ij} = 1$ (relevance) if the lyric at rank j expresses the same emotion as the prompt, and $rel_{ij} = 0$ otherwise. The term $|R_i|$ denotes the number of emotionally relevant lyrics for prompt i. The nDCG metric is bounded between 0 and 1, with higher values indicating better emotionally aligned rankings. The discount gives lower weight to items ranked lower, while the IDCG (ideal DCG) represents the maximum achievable DCG with the same set of relevance scores but in the perfect ranking order, thus normalizing the metric.

- **BLEU@k**: provides a standard similarity metric between prompt and lyrics. This is not expected to yield good results as the prompt and the lyrics have drastically different lenghts, but is included for completeness.

Table 3a: Explicit prompts used for evaluation, with emotion and the clear affective signals

Prompt	**Emotion**	**Clear affective signals**
I feel so empty inside	Sadness	**empty inside**
I can't stop crying tonight	Sadness	**crying**
My heart aches for what's gone	Sadness	**heart aches**
Nothing feels worth it anymore	Sadness	**nothing, worth it**
I'm surrounded by silence and pain	Sadness	**silence**, **pain**
I feel pure happiness today	Joy	**happiness**
Everything about this day makes me smile	Joy	**smile**
My heart is bursting with excitement	Joy	**excitement**
I'm blessed to be alive	Joy	**blessed**, **alive**
I've never been this happy in my life	Joy	**happy**
I'm terrified of what might happen	Fear	**terrified**
I feel panic rising in my chest	Fear	**panic rising**
I'm afraid to be alone right now	Fear	**afraid**
I'm scared I might not be enough	Fear	**scared**
I can't breathe; the fear is suffocating	Fear	**fear**, **can't breathe**
I'm furious about what happened	Anger	**furious**
This makes my blood boil from outrage	Anger	**outrage**
I can't stand being lied to	Anger	**can't stand**, **lied to**
I'm sick of being ignored	Anger	**sick of**, **ignored**
I'm filled with rage	Anger	**rage**

Table 3b: Implicit prompts used for evaluation, with inferred emotion and the key entity (3Q: *What*).

Prompt	Emotion	Entity
I just left my girlfriend at the airport, and I am not going to see her for 3 months	Sadness	Girlfriend
The coffee's gone cold again	Sadness	Cold coffee
I deleted our photos yesterday	Sadness	Photos
The sky has been gray for days and it's never going to clear	Sadness	Sky
I made a lot of mistakes I don't know how to forgive myself	Sadness	Mistakes
This summer I'm going on vacation with my friends	Joy	Vacation
I finally feel my life has meaning	Joy	Life
The air feels lighter somehow	Joy	Air
The room feels warmer when they are here	Joy	Room
My dreams have finally come true	Joy	Dreams
The walls feel like they are closing in	Fear	Walls
Every sound makes me jump	Fear	Sound
The future feels like a storm I can't outrun	Fear	Future
I keep the lights on at night to avoid the dark	Fear	Lights
I feel like I'm in a dark tunnel and don't know how to get out	Fear	Tunnel
They took the credit again	Anger	Credit
I bit my tongue so hard it almost bled	Anger	Tongue
All I want to do is punch the wall	Anger	Wall
Our political system is unbelievable	Anger	Political system
I'm done with the hypocrisy of this world	Anger	Hypocrisy

4.2.2 Evaluation of generation

An evaluation of the generation capabilities was conducted to assess the quality of the system on three main aspects: groundedness, consistency and adaptability. Regarding groundedness, starting from the prompts introduced in section 4.2.1, each LLM was queried providing a total of 40 responses each. Subsequently, the true candidates' list available to the model at inference time was extracted and compared with the songs that were actually recommended; these were detected using regex patterns constructed

by manually inspecting the sample responses (in order to check possible divergences from the output format in Table 6) and subsequent manual checks. Lastly, the ratio of hallucinated suggestions was computed, allowing to check whether the models were actually exploiting the evidence from the knowledge base.

As a second step, to assess consistency, each LLM was queried with the previous prompts multiple times, thus obtaining 10 responses per prompt, for a total of 1200 responses. Each response was then embedded using a simple transformer-based model (`all-MiniLM-L6-v2`, [48]), obtaining a single dense vector per response. Finally, a custom metric was implemented allowing computation of pairwise similarity among vectors related to the same prompts for a total of 45 similarity scores per prompt, which were then averaged across prompts. This metric yields a single value for each LLM, with higher scores suggesting better consistency across answers. Formally the process is described as follows: Let

- $P = \{p_1, p_2, \ldots, p_{N_p}\}$ be the set of prompts (with $N_p = 40$),
- R_{ij} denote the j^{th} response ($j = 1, \ldots, 10$) generated by an LLM for prompt p_i,
- $\mathbf{E}(\mathbf{R_{ij}}) \in \mathbb{R}^d$ be the embedding vector of R_{ij}, obtained via the transformer-based embedding model,
- $sim(\mathbf{E}(\mathbf{R_{ij}}), \mathbf{E}(\mathbf{R_{ik}}))$ be the similarity function between two embeddings (e.g., cosine similarity, Equation (2)).

Then, for each prompt p_i, the **average pairwise similarity** is computed as:

$$S_i = \frac{2}{n(n-1)} \sum_{1 \leq j < k \leq n} sim(\mathbf{E}(\mathbf{R_{ij}}), \mathbf{E}(\mathbf{R_{ik}})) \tag{5}$$

where $n = 10$ responses per prompt, leading to $\binom{10}{2} = 45$ pairwise comparisons.
Finally, the **consistency score** for the LLM across all prompts is the mean of these prompt-level similarities:

$$S_{\text{LLM}} = \frac{1}{N_p} \sum_{i=1}^{N_p} S_i \tag{6}$$

Lastly, in an attempt to test the model's adaptability, a stress-test is implemented. While this study primarily focuses on a system that retrieves based on the semantic meaning extracted from song lyrics, some preliminary analysis showed that prompting the model

to suggest songs by a specific artist actually yielded surprising and promising results. In fact, this is likely due to the structure of the lyrics themselves on the Genius platform, as the artist is often cited at the start of a verse or a hook. This pattern holds especially true for songs where said artist is featured or when they are not the main artist, since Genius tends to specify the contributors of each block for this kind of tracks. Hence, the proposed procedure to test the model's adaptability to other tasks is to construct a new set of prompts, where the model is asked to provide recommendations for tracks that are performed by a specific artist. The LLM is then tested based on how many of the relevant songs that were retrieved, are actually included in the final recommendation. Ideally, if the retriever returns 3 songs out of 20 for the required artist, all of them should be included in the recommendation, independently of their rank. The process can be described as follows:

- Let $\mathcal{A} = \{a_1, a_2, \ldots, a_{N_a}\}$ denote the set of all artists in the dataset.
- For each artist $a_i \in \mathcal{A}$, let s_{a_i} be the total number of available songs in the candidate pool.
- Artists with fewer than twenty available songs were excluded to facilitate retrieval, yielding the filtered set

$$\mathcal{A}' = \{a_i \in \mathcal{A} \mid s_{a_i} \geq 20\}. \tag{7}$$

- The remaining artists were divided into Q disjoint quantile intervals based on s_{a_i}. Let the quantile boundaries be denoted as

$$q_0 < q_1 < \cdots < q_Q,$$

  where each interval $(q_{k-1}, q_k]$ defines a distinct non-overlapping group of artists:

$$\mathcal{A}_k = \{a_i \in \mathcal{A}' \mid q_{k-1} < s_{a_i} \leq q_k\}, \quad k = 1, \ldots, Q. \tag{8}$$

- From each quantile group $\mathcal{A}_k$, eleven distinct artists were randomly sampled:

$$\mathcal{S}_k \subset \mathcal{A}_k, \quad |\mathcal{S}_k| = 11. \tag{9}$$

- The final evaluation set was defined as the union of these disjoint samples:

$$\mathcal{S} = \bigcup_{k=1}^{Q} \mathcal{S}_k. \tag{10}$$

- For each artist $a \in \mathcal{S}$, the LLM was prompted to suggest songs by that artist.

- Let c_a denote the number of the candidate songs that were retrieved for artist a and r_a the number of songs that were actually recommended.

- The **success rate** for each artist is computed as:

$$SR = \frac{r_a}{c_a} \times 100, \tag{11}$$

- The overall **adaptability score** for the LLM was obtained by averaging across all sampled artists:

$$SR_{\text{LLM}} = \frac{1}{|\mathcal{S}|} \sum_{a \in \mathcal{S}} r_a. \tag{12}$$

5 Results

This section reports the experimental findings obtained from evaluating the proposed Retrieval-Augmented Generation (RAG) framework for lyric-based music recommendation. The analysis covers three aspects: discussion of generated responses, quantitative assessment of emotional alignment between user queries and retrieved lyrics, and evaluation of the generation stage in terms of groundedness, consistency, and adaptability.

5.1 RAG

After repeated interactions with the system, it emerges that generated responses closely align with the intended behavior outlined in the system prompt. The model consistently produces five recommendations, adhering to the requested structure and stylistic tone of a "friendly music curator". Its responses demonstrate the ability to integrate the available information, particularly by referencing relevant lyrical content and implicitly leveraging audio and annotation features to justify the song selection, as it shows emotional and thematic coherence in its choices. The explanations for each track are concise, varied, and emotionally aligned, reflecting the instruction to adapt recommendations to the user's perceived emotion without explicitly mentioning technical attributes such as valence or energy. Minor deviations from the expected output format occasionally occur, most notably in LLaMA's generations, which sometimes introduce additional line breaks or vary slightly in phrasing; however, these do not affect semantic fidelity or recommendation quality. Overall, the system displays a satisfactory performance by balancing the factual grounding from the retrieved candidates with empathetic framing provided by the LLMs. Moreover, it is able to maintain structural and emotional consistency across prompts, which indicates a solid adherence to the task's constraints and successful conditioning on the system prompt across different generative models. Regarding major differences in generation style, Mistral and LLaMA tend to produce more creative and contextually rich responses, often framing the playlist within a narrative context, whereas Qwen provides more direct outputs, focusing on listing suitable tracks without extensive commentary. Figure 8 reports an example interaction with the system, using LLaMA3 as a generator.

Example interaction generated through LLaMA3 - 7b

Prompt: *I just left my girlfriend at the airport, and I am not going to see her for 3 months*

Response: I understand that you're going through a tough time, and my goal is to help you find some comfort in music. After reviewing the candidates, I've curated a list of the top 5 songs that I think will resonate with your emotions.

Here are my picks:

1) **(Lost Her Love) On Our Last Date – Conway Twitty**
This country classic speaks directly to your heartache and longing. The lyrics capture the pain of being apart from someone you love, and the song's gentle melody provides a soothing balm for your soul.

2) **I'll Wait for You – Joe Nichols**
This song is a beautiful expression of devotion and commitment. Even though you're physically apart, the singer promises to wait patiently for his loved one to return. The country-pop soundscapes will transport you to a place of hope and longing.

3) **Tender Hearted Baby – George Hamilton IV**
This 1960s country tune has a timeless quality that will speak to your emotions. The lyrics convey the pain of being away from someone you love, and the song's gentle, swaying rhythm will provide a comforting backdrop for your thoughts.

4) **How Soon – Joe**
This R&B ballad is a poignant expression of the pain of separation. The singer's soulful vocals and the song's lush instrumentation create a sense of intimacy and longing that will resonate deeply with your emotions.

5) **This God Damn House – The Low Anthem**
This indie-folk song has a haunting beauty that will speak to your feelings of isolation and loneliness. The lyrics capture the pain of being apart from someone you love, and the song's soaring chorus will provide a sense of release and catharsis.

These songs are not just great music; they're also a reflection of the human experience. They understand the pain of heartache and the longing for connection. I hope these picks bring you some comfort during this difficult time.

Figure 8: Example of model-generated playlist response to an emotional prompt.

5.2 Emotional evaluation

The emotional evaluation aimed to assess whether the Retrieval-Augmented system was capable of selecting lyrics whose emotion aligns with the affective intent of user queries. The test set comprised 40 manually constructed prompts divided between explicit prompts (emotion lexically stated) and implicit prompts (emotion inferred through entity and situational context), covering four target emotions: joy, sadness, anger, and fear (Table 3a and Table 3b). Each prompt was used to query the retriever, which returned the top 50 candidate songs. The emotion of each retrieved corpus was automatically classified using the multilingual emotion recognition model `MilaNLProc/xlm-emo-t`, mapping each text

to one of the four emotion categories. Evaluation metrics defined in Section 4.2.1 were then computed to quantify how well the retrieved emotions matched the target prompt emotion.

5.2.1 Overall metrics

The aggregate results (Table 4) show that the retriever consistently ranks emotionally relevant lyrics near the top of the candidate list. Overall, the high Mean Reciprocal Rank ($MRR@10 \approx 0.78$) suggests that the first relevant result is usually ranked 1st or 2nd, while the Normalized Discounted Cumulative Gain ($nDCG@10 \approx 0.80$) confirms that relevant items generally appear closer to the top of the list and that ranking quality remains stable even as the number of retrieved items increases to k = 50. The moderate precision and recall values (68% and 59%, respectively) indicate a decent performance in retrieving emotionally matching lyrics. However, by increasing the number of retrieved results (k), a drop in performance is observed. This result is to be expected and confirms that the ranking provided by the retriever is consistent with relevance.
Regarding explicit prompts, results are much better across all metrics showing similar patterns as k increases. In particular, a higher F1-score is observed (by roughly 10 percentage points), suggesting overall better matches in the results.
Conversely, results for implicit prompts are unfortunately worse, showing lower accuracy metrics (on average 60% precision, 50% recall and 46% F1-score). The MRR@k however, shows that the first relevant result is usually ranked in the top two positions (on average), with the nDCG confirming the presence of relevant results near the top the list.

Finally, the very low BLEU@k values across the board ($= 0.003 - 0.004$) are justified by the different lengths of prompts and lyrics. Relatively speaking, however, BLEU scores are higher for the top results, showing the model's ability to retrieve similar text, with explicit lexical expression being more effective than implicit situational framing.

5.2.2 Emotion-wise metrics

While overall metrics may help to describe the performance of the system, a more detailed analysis is required in order to unpack the contribution of each emotion to these metrics. Results are reported in Tables 5a through 5d. It is worth noting that in this evaluation

Table 4: Retriever performance by prompt type (Explicit, Implicit, Overall) across retrieval depths $k \in \{5, 10, 20, 50\}$. Metrics are averaged across emotions.

	Explicit						Implicit						Overall					
k	**MRR**	**Prec**	**Rec**	**F1**	**nDCG**	**BLEU**	**MRR**	**Prec**	**Rec**	**F1**	**nDCG**	**BLEU**	**MRR**	**Prec**	**Rec**	**F1**	**nDCG**	**BLEU**
5	0.85	0.77	0.69	0.68	0.88	0.005	0.68	0.60	0.48	0.44	0.68	0.004	0.76	0.69	0.59	0.57	0.78	0.004
10	0.86	0.76	0.69	0.68	0.88	0.004	0.69	0.61	0.50	0.48	0.71	0.004	0.78	0.68	0.60	0.58	0.80	0.004
20	0.86	0.74	0.69	0.68	0.88	0.003	0.70	0.60	0.50	0.47	0.73	0.003	0.78	0.67	0.59	0.58	0.80	0.003
50	0.86	0.72	0.65	0.64	0.87	0.003	0.70	0.61	0.49	0.45	0.74	0.003	0.78	0.66	0.57	0.55	0.80	0.003

setting, accuracy replaces precision, recall, and F1-score, as these metrics become uninformative when the true label is constant. Since all retrieved items are evaluated against a single target emotion per prompt, there are no true negatives, making the standard confusion-matrix–based measures meaningless.

For **Joy**, we can see from Table 5a, implicit prompts surprisingly show higher MRR (0.90) and accuracy than explicit (0.80) across all k. This result is unexpected, and could be explained by the semantic structure of the sampled songs themselves, which might express joy through imagery more often than lexical description. Nevertheless, it shows that the retriever is able to consistently rank joyful songs in the top spot for joyful prompts. Accuracy shows better results for explicit prompts, and it is surprisingly increasing in k; this phenomenon shows that correct results "accumulate" faster than incorrect ones as k increases, meaning that 50 items are not a sizeable enough requirement to hinder the retriever's ability to find joyful songs. nDCG steadily improves with k in all splits further confirming this hypothesis, peaking at 0.94 (explicit) and 0.92 (overall), indicating that relevant joyful songs are ranked near the top and become more concentrated deeper in the list.

Regarding **Sadness**, the retriever is always able to place a relevant result in the top spot ($MRR@k = 1, \forall k$), both for explicit and implicit prompts. Moreover, as for joy, accuracy is lower for implicit prompts but increasing in k; it is however the most accurately retrieved emotion. As for the nDCG, values are solid and above 0.90 across the board reflecting the ability of the retriever to correctly rank sad songs; in contrast with Joy, however, results show that as the number of retrieved items increases, performance slightly decays with some irrelevant items beating relevant ones.

Concerning **Anger**, the results are diverging from the previous two emotions. In fact, both the MRR and the accuracy are much lower, showing that the retriever is struggling to capture the emotion in retrieved songs. While on average a relevant result is always provided among the top two positions, the overall set of items is lacking, with accuracy

lower than 40% in most cases. Moreover, the pattern is inconsistent as k increases, showing that, differently from *Joy*, the number of items after which correct retrieval starts to decay is lower. (After 20 items it seems that the proportion of correct results increases less than incorrect ones). The nDCG follows a similar logic, with lower performance compared to the previous emotions. Nevertheless, results for explicit and implicit prompts are more aligned than for Joy and Sadness, showing that songs that portray anger, often do so in explicit language rather than imagery.

Lastly, for **Fear**, the results for explicit prompts drastically outclass the ones for implicit prompts. It is evident that the retriever is unable to fetch fear-centered songs without explicit lexical indications, and it cannot infer the emotion from the entity or the situation. This is confirmed by the MRR being perfect for explicit prompts and incredibly low for implicit ones. The same pattern holds for accuracy and the nDCG; both the former and the latter decrease in k implying that as the number of retrieved items increases, less and less fear-related songs are ranked. Such an outcome is likely due to the similarity of this emotion to sadness in implicit contexts.

Finally, the BLEU score seems to follow a similar pattern the the overall case, with negligible scores (due to the differences in length), but decreasing pattern as k increases and generally higher for explicit prompts, showing that more similar results are correctly placed near the top of the list, and clear lexical descriptions improve this metric.

Table 5a: Retriever performance for **Joy** with accuracy (Acc)

	Explicit				**Implicit**				**Overall**			
k	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**
5	0.80	0.80	0.89	0.007	0.90	0.68	0.89	0.002	0.85	0.74	0.89	0.005
10	0.80	0.86	0.90	0.005	0.90	0.72	0.88	0.002	0.85	0.79	0.89	0.004
20	0.80	0.86	0.92	0.004	0.90	0.72	0.88	0.002	0.85	0.79	0.90	0.004
50	0.80	0.87	0.94	0.003	0.90	0.77	0.90	0.002	0.85	0.82	0.92	0.003

Table 5b: Retriever performance for **Sadness** with accuracy (Acc)

	Explicit				**Implicit**				**Overall**			
k	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**
5	1.00	0.84	0.98	0.003	1.00	0.80	0.95	0.003	1.00	0.82	0.96	0.003
10	1.00	0.82	0.96	0.003	1.00	0.78	0.94	0.003	1.00	0.80	0.95	0.003
20	1.00	0.85	0.96	0.003	1.00	0.75	0.93	0.003	1.00	0.80	0.94	0.003
50	1.00	0.83	0.95	0.003	1.00	0.72	0.92	0.003	1.00	0.77	0.94	0.003

Table 5c: Retriever performance for **Anger** with accuracy (Acc)

	Explicit				Implicit				Overall			
k	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**
5	0.60	0.36	0.66	0.005	0.57	0.28	0.58	0.005	0.58	0.32	0.62	0.005
10	0.62	0.38	0.71	0.004	0.57	0.32	0.57	0.004	0.59	0.35	0.64	0.003
20	0.62	0.42	0.70	0.003	0.58	0.33	0.61	0.004	0.60	0.38	0.66	0.005
50	0.62	0.37	0.71	0.002	0.58	0.33	0.64	0.003	0.60	0.35	0.68	0.003

Table 5d: Retriever performance for **Fear** with accuracy (Acc)

	Explicit				Implicit				Overall			
k	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**	**MRR**	**Acc**	**nDCG**	**BLEU**
5	1.00	0.76	0.97	0.004	0.25	0.16	0.30	0.005	0.63	0.46	0.64	0.005
10	1.00	0.70	0.95	0.004	0.31	0.20	0.48	0.005	0.66	0.45	0.71	0.004
20	1.00	0.61	0.92	0.004	0.32	0.18	0.52	0.004	0.66	0.40	0.72	0.004
50	1.00	0.52	0.89	0.003	0.32	0.14	0.50	0.003	0.66	0.33	0.69	0.003

5.3 Evaluation of generation

5.3.1 Groundedness

Groundedness was evaluated to verify whether the generated song recommendations were supported by the evidence retrieved from the knowledge base. Using the procedure detailed in Section 4.2.2, each model's forty responses were processed through a set of regular expressions constructed from manual inspection of sample outputs, allowing an automatic extraction for both song titles and artist names, which was followed by a thorough manual inspection to confirm soundness. The extracted information was then compared with the candidate lists that were available to the models at inference time. The analysis revealed a complete alignment between the generated recommendations and the retrieved evidence: all models produced responses that exclusively referenced songs and artists present in the candidate lists, with no instances of hallucinated items. Manual checks confirmed that even when formatting variations occurred, the factual content of the outputs remained accurate and traceable to the underlying knowledge base. This finding indicates that the generation stage was consistently grounded in retrieval results, thus showing a stable integration between the retriever and generator components. However, as will be discussed in Section 5.3.3, when the models were prompted with tasks that deviated from the original setup, specifically in the adaptability stress test, some degree of hallucination emerged, reflecting the increased difficulty of maintaining groundedness under altered

prompt conditions.

5.3.2 Consistency

The consistency analysis aimed to measure how stably each language model reproduced semantically similar answers when exposed repeatedly to identical prompts, providing a measure of generative variance. As reported in Table 6, all models achieved high levels of internal agreement, with cosine similarity scores ranging from approximately 0.84 and 0.88 across prompt types. Among them, Mistral displayed the strongest overall consistency (0.876), followed closely by Qwen (0.869) and LLaMA (0.845). The same relative ranking was confirmed by the BLEU metric, which captures more surface-level lexical overlap, with Qwen yielding the highest BLEU values (31%), indicating the most uniform phrasing across generations. Conversely, LLaMA's lower BLEU (21%) suggests lower linguistic stability, even though its semantic similarity remains high. Overall, these results show that all three models maintain a coherent latent representation of the responses, but Mistral achieves the most stable semantic encoding, while Qwen produces the most lexically repetitive responses. These results were reinforced by a manual inspection, which revealed a worse adherence to the requested format for LLaMA, with higher variability in the lexical structure of the response. Qwen and Mistral however, rarely displayed diverging formats suggesting they might be able to better interpret the system prompt. Lastly, a small and consistent difference between implicit and explicit prompts is observed across models, with more straightforward prompts contributing to a slight reduction in the variance of the responses.

	Explicit		**Implicit**		**Overall**	
	Cosine Similarity	**BLEU**	**Cosine Similarity**	**BLEU**	**Cosine Similarity**	**BLEU**
LLaMA3 - 7b	0.84	21.5%	0.84	21.1%	0.85	21.3%
mistral - 7b	0.88	27.1%	0.87	26.7%	0.88	26.9%
Qwen2.5 - 7b	0.87	31.8%	0.87	31.0%	0.87	31.4%

Table 6: Average pairwise cosine similarity and BLEU scores by model and prompt type.

5.3.3 Adaptability

Adaptability was evaluated through the stress-test described in Section 4.2.2, where each model was required to deviate from the system prompt and adapt to provide an artist-specific recommendation. Immediately, one major limitation inherent to this approach is

that the LLM relies on the retriever to provide relevant results (i.e. songs by the requested artist), which amounts to roughly half the prompts in the sample (56). While this reduces the total cases in which the adaptability of the model can be assessed, it allowed for the extraction of valuable insights in terms of hallucinations; specifically, two hallucination rates were computed for the cases in which the model either recommended out of sample songs or recommended a song from a different artist implying that it belonged to the request one. Two different rates were computed:

- **Hallucination Rate (Hits)**: Rate of hallucinations among prompts for which the retriever yielded relevant results.
- **Hallucination Rate (Misses)**: Rate of hallucinations among prompts for which the retriever did not yield any relevant result.

The results in Table 7 reveal clear differences in the models' ability to adjust their recommendation behavior. Mistral achieved the highest adaptability score (93.2%), indicating a strong propensity to exploit the altered prompt condition and identify relevant songs by the requested artist. Qwen followed with 89.2%, while LLaMA reached 82.5%, suggesting a comparatively lower level of prompt sensitivity. However, this adaptability in Mistral was accompanied by a significantly higher negative hallucination rate (54.7%), implying that its flexibility sometimes came at the cost of precision, generating song recommendations that were not grounded in the candidate list. In fact, through manual inspection, it emerged that in many cases when Mistral had no relevant candidates to choose from, it defaulted to its own training data, often recommending out-of-sample, albeit correct songs. In contrast, Qwen combined high adaptability with very low hallucination rates (3.5% positive, 1.9% negative), demonstrating a more balanced capability to adjust to the new task while preserving factual reliability. Overall, these findings suggest that while all models exhibit some adaptability to a different task, Qwen provides the better trade-off in terms of responsiveness and groundedness, whereas Mistral prioritizes flexibility at the expense of accuracy.

Model	Adaptability Score	Hallucination Rate (Hits)	Hallucination Rate (Misses)
LLaMA3 - 7b	82.5%	5.3%	5.7%
mistral - 7b	93.2%	7.0%	54.7%
Qwen2.5 - 7b	89.2%	3.5%	1.9%

Table 7: Adaptability scores and hallucination rates for prompts for which the retriever did (Hits) or did not (Misses) returned results

6 Limitations and Further Analysis

Although the proposed architecture shows promising results in relevance, grounding, consistency, and adaptability, several limitations should be acknowledged, as well as potential directions for further analysis.

First, while the system is capable of providing highly personalized song suggestions, this personalization still depends on the user's ability to articulate their emotional and mental state explicitly through natural language prompts. In contrast, algorithmic recommenders such as those employed in large-scale streaming platforms infer user preferences implicitly from behavioral data (e.g., listening history, skip rate or session length). Consequently, the system assumes a relatively high level of self-awareness and expressiveness from the user, which may limit practical adoption in real-world settings.

Second, the system architecture is deliberately simple, focusing on the interaction between retrieval and generation. Future studies could improve the pipeline by introducing additional components such as specialized LLM reranking layers (with possible voting mechanisms), genre and artist recognition modules, or filtering layers for mood, tempo, or explicit content. Thus, a more "agentic" approach to this task could yield better results improving both relevance and consistency.

Third, the use of open-source language models ensures reproducibility and accessibility but inevitably reduces performance compared to the state of the art. Employing larger and more capable models, such as those available through OpenAI's API, would likely enhance the quality of recommendations, albeit at the expense of openness and economic costs.

A further limitation lies in the evaluation of the generative component, which required extensive manual inspection to verify groundedness and to interpret qualitative differences in responses. While this approach ensured accuracy and soundness, it introduces an element of subjectivity and non-replicability into the assessment process. Further studies should focus on the implementation of more objective measures to overcome these inherent issues.

Furthermore, the dataset coverage remains incomplete, as a substantial portion of songs in the source corpus lack Genius' annotations. This sparsity can limit the amount of information that the system includes in the final recommendation, necessarily resulting in a worse performance. Future research should focus on expanding and enriching the

annotated dataset, exploring ways to scale the sample size.
Beyond these methodological aspects, two broader limitations should be noted. Firstly, the current study does not include a human-centered evaluation of recommendation satisfaction or perceived relevance, which would be essential for assessing the system's practical value in real use cases. Secondly, while international songs appear in the sample and can be recommended, the present setup focuses primarily on English content, leaving open the question of how well the model generalizes across languages and cultural contexts. While the multilingual embedder used to construct the vector database may partially overcome this concern, it should be taken into consideration in further studies.

Lastly, these limitations do not undermine the validity of the results, which are indeed promising, but rather they help to create a roadmap for future research. They highlight the trade-offs inherent in balancing openness, reproducibility, and performance, showcasing clear opportunities for improvements. Addressing these constraints will potentially allow the system to evolve from an experimental prototype into a more scalable recommendation framework applicable to real-world contexts.

7 Conclusion

This thesis presented a novel framework for lyric-based music recommendation grounded in a Retrieval-Augmented Generation (RAG) architecture. The system integrates dense semantic retrieval with large language model generation to provide emotion-aware song suggestions. Through the creation of a new dataset that encompasses lyrics, annotations, and audio features for more than 220,000 tracks, this study contributes both a methodological and empirical foundation for future research in the context of Natural Language Processing and music information retrieval.

The experimental results demonstrate that the system is capable of capturing the emotional intent of the user, retrieving and generating musically and affectively relevant recommendations. Evaluation of the generation stage revealed a high degree of groundedness across all models, with no hallucinated content under standard conditions, implying a strong alignment between the retrieved evidence from the knowledge base and the generated output. A consistency analysis showed that the models produced semantically and structurally stable responses, while the adaptability stress-test indicated that the architecture could generalize to modified tasks such as artist-specific recommendation, albeit at the cost of occasional hallucinations when retrieval support was limited.

Beyond the empirical outcomes, this thesis supports the broader potential of RAG-based frameworks in music-related applications. The system manages to maintain reliability through grounded retrieval and to enable personalized, linguistically expressive interactions; these, in contrast to standard algorithmic recommendation, offer an actively stimulating experience for music discovery. However, several challenges remain, such as the dependence on user articulation, the limited scalability of manual evaluation, and the partial coverage of annotated data.

Overall, the results presented in this study validate the conceptual soundness of the proposed architecture and provide a solid foundation for future development. Extending this work toward multimodal retrieval, more powerful generative models, and more automated evaluation methods could yield significant improvements in both performance and generalization.

References

[1] G. Adomavicius and A. Tuzhilin. "Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions". In: *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), pp. 734–749. DOI: 10.1109/TKDE.2005.99.

[2] Tamim Mahmud Al-Hasan et al. "From Traditional Recommender Systems to GPT-Based Chatbots: A Survey of Recent Developments and Future Directions". In: *Big Data and Cognitive Computing* 8.4 (2024), p. 36. DOI: 10.3390/bdcc8040036. URL: https://doi.org/10.3390/bdcc8040036.

[3] Amazon Web Services. *Amazon Bedrock Knowledge Bases: Retrieval-Augmented Generation for Your Data*. https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html. 2025.

[4] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2016. arXiv: 1409.0473 [cs.CL]. URL: https://arxiv.org/abs/1409.0473.

[5] Matej Bevec, Marko Tkalčič, and Matevž Pesek. "Hybrid music recommendation with graph neural networks". In: *User Modeling and User-Adapted Interaction* 34.5 (Aug. 2024), 1891–1928. ISSN: 0924-1868. DOI: 10.1007/s11257-024-09410-4. URL: https://doi.org/10.1007/s11257-024-09410-4.

[6] Federico Bianchi, Debora Nozza, and Dirk Hovy. "XLM-EMO: Multilingual Emotion Prediction in Social Media Text". In: *Proceedings of the 12th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*. Association for Computational Linguistics, 2022.

[7] Lorenz Brehme et al. *Retrieval-Augmented Generation in Industry: An Interview Study on Use Cases, Requirements, Challenges, and Evaluation*. 2025. arXiv: 2508.14066 [cs.IR]. URL: https://arxiv.org/abs/2508.14066.

[8] Oscar Celma. *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space*. 1st. Springer Publishing Company, Incorporated, 2010. ISBN: 3642132863.

[9] Harrison Chase. *LangChain: A framework for developing applications powered by large language models*. Open-source software; accessed Nov 2025. 2022. URL: https://github.com/hwchase17/langchain.

[10] Jianlv Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: 2402.03216 [cs.CL]. URL: https://arxiv.org/abs/2402.03216.

[11] Wenhu Chen et al. "MuRAG: Multimodal Retrieval-Augmented Generator for Open Question Answering over Images and Text". In: (2022). arXiv: 2210.02928 [cs.CL]. URL: https://arxiv.org/abs/2210.02928.

[12] Wei Cheng et al. "Prompt Memoried LLMs with 'What, Why, and How' to Reason Implicit Emotions". In: *ICCBR Workshops*. 2024, pp. 26–38. URL: https://ceur-ws.org/Vol-3708/paper_03.pdf.

[13] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". In: *arXiv preprint arXiv:1406.1078* (2014). URL: https://arxiv.org/abs/1406.1078.

[14] Renato Cunha, Evandro Caldeira, and Luciana Fujii. "Determining Song Similarity via Machine Learning Techniques and Tagging Information". In: (Apr. 2017). DOI: 10.48550/arXiv.1704.03844.

[15] Felix Darke and Linus Below Blomkvist. *Categorization of songs using spectral clustering*. 2021.

[16] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[17] Seungheon Doh, Keunwoo Choi, and Juhan Nam. *TALKPLAY: Multimodal Music Recommendation with Large Language Models*. 2025. arXiv: 2502.13713 [cs.IR]. URL: https://arxiv.org/abs/2502.13713.

[18] Matthijs Douze et al. *The Faiss library*. 2025. arXiv: 2401.08281 [cs.LG]. URL: https://arxiv.org/abs/2401.08281.

[19] Charbel El Achkar. "Music encoding and deep learning for music transcription and classification based on visually represented audio features". Theses. Université Bourgogne Franche-Comté, Dec. 2023. URL: https://theses.hal.science/tel-04483299.

[20] Maria Eriksson et al. *Spotify Teardown: Inside the Black Box of Streaming Music*. The MIT Press, Feb. 2019. ISBN: 9780262349680. DOI: 10.7551/mitpress/10932.001.0001. URL: https://doi.org/10.7551/mitpress/10932.001.0001.

[21] Zhouyu Fu et al. "A Survey of Audio-Based Music Classification and Annotation". In: *Multimedia, IEEE Transactions on* 13 (May 2011), pp. 303 –319. DOI: 10.1109/TMM.2010.2098858.

[22] Tianyu Gao, Xingcheng Yao, and Danqi Chen. *SimCSE: Simple Contrastive Learning of Sentence Embeddings*. 2022. arXiv: 2104.08821 [cs.CL]. URL: https://arxiv.org/abs/2104.08821.

[23] Google Cloud. *Grounding overview*. https://cloud.google.com/vertex-ai/generative-ai/docs/grounding/overview. Google Cloud, 2025. URL: https://cloud.google.com/vertex-ai/generative-ai/docs/grounding/overview.

[24] Kelvin Guu et al. "REALM: Retrieval-Augmented Language Model Pre-Training". In: (2020). arXiv: 2002.08909 [cs.CL]. URL: https://arxiv.org/abs/2002.08909.

[25] Haoyu Han et al. "Retrieval-Augmented Generation with Graphs (GraphRAG)". In: (2025). arXiv: 2501.00309 [cs.IR]. URL: https://arxiv.org/abs/2501.00309.

[26] David Hesmondhalgh. "Streaming's Effects on Music Culture: Old Anxieties and New Simplifications". In: *Cultural Sociology* 16.1 (2022), pp. 3–24. DOI: 10.1177/17499755211019974. eprint: https://doi.org/10.1177/17499755211019974. URL: https://doi.org/10.1177/17499755211019974.

[27] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[28] Dietmar Jannach et al. "A Survey on Conversational Recommender Systems". In: *ACM Computing Surveys* 54.5 (May 2021), 1–36. ISSN: 1557-7341. DOI: 10.1145/3453154. URL: http://dx.doi.org/10.1145/3453154.

[29] Albert Q. Jiang et al. *Mistral 7B*. 2023. arXiv: 2310.06825 [cs.CL]. URL: https://arxiv.org/abs/2310.06825.

[30] Vladimir Karpukhin et al. *Dense Passage Retrieval for Open-Domain Question Answering*. 2020. arXiv: 2004.04906 [cs.CL]. URL: https://arxiv.org/abs/2004.04906.

[31] Peter Knees and Markus Schedl. "Collaborative Music Similarity and Recommendation". In: *Music Similarity and Retrieval: An Introduction to Audio- and Web-based Strategies*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016, pp. 179–211. ISBN: 978-3-662-49722-7. DOI: 10.1007/978-3-662-49722-7_8. URL: https://doi.org/10.1007/978-3-662-49722-7_8.

[32] Yehuda Koren, Robert Bell, and Chris Volinsky. "Matrix Factorization Techniques for Recommender Systems". In: *Computer* 42.8 (2009), pp. 30–37. DOI: 10.1109/MC.2009.263.

[33] Dominik Kowald, Markus Schedl, and Elisabeth Lex. "The Unfairness of Popularity Bias in Music Recommendation: A Reproducibility Study". In: *CoRR* abs/1912.04696 (2019). arXiv: 1912.04696. URL: http://arxiv.org/abs/1912.04696.

[34] Daeyong Kwon, SeungHeon Doh, and Juhan Nam. *MUST-RAG: MUSical Text Question Answering with Retrieval Augmented Generation*. 2025. arXiv: 2507.23334 [cs.CL]. URL: https://arxiv.org/abs/2507.23334.

[35] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: (2021). arXiv: 2005.11401 [cs.CL]. URL: https://arxiv.org/abs/2005.11401.

[36] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. 2013. arXiv: 1301.3781 [cs.CL]. URL: https://arxiv.org/abs/1301.3781.

[37] Armin Moradi, Nicola Neophytou, and Golnoosh Farnadi. "Advancing Cultural Inclusivity: Optimizing Embedding Spaces for Balanced Music Recommendations". In: (2024). arXiv: 2405.17607 [cs.IR]. URL: https://arxiv.org/abs/2405.17607.

[38] Jeremy Morris. "Music Platforms and the Optimization of Culture". In: *Social Media + Society* 6 (July 2020), p. 205630512094069. DOI: 10.1177/2056305120940690.

[39] Skatje Myers et al. "Lessons learned on information retrieval in electronic health records: a comparison of embedding models and pooling strategies". In: *Journal of the American Medical Informatics Association* 32.2 (Dec. 2024), 357–364. ISSN: 1527-974X. DOI: 10.1093/jamia/ocae308. URL: http://dx.doi.org/10.1093/jamia/ocae308.

[40] Enrico Palumbo et al. "Text2Tracks: Prompt-based Music Recommendation via Generative Retrieval". In: (2025). arXiv: 2503.24193 [cs.IR]. URL: https://arxiv.org/abs/2503.24193.

[41] Jeffrey Pennington, Richard Socher, and Christopher Manning. "GloVe: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Alessandro Moschitti, Bo Pang, and Walter Daelemans. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1532–1543. DOI: 10.3115/v1/D14-1162. URL: https://aclanthology.org/D14-1162/.

[42] Robert Prey. "Locating Power in Platformization: Music Streaming Playlists and Curatorial Power". In: *Social Media + Society* 6 (Apr. 2020), p. 205630512093329. DOI: 10.1177/2056305120933291.

[43] Robert Prey. "Nothing personal: algorithmic individuation on music streaming platforms". In: *Media, Culture & Society* 40.7 (2018). PMID: 30270951, pp. 1086–1100. DOI: 10.1177/0163443717745147. eprint: https://doi.org/10.1177/0163443717745147. URL: https://doi.org/10.1177/0163443717745147.

[44] Qwen et al. *Qwen2.5 Technical Report*. 2025. arXiv: 2412.15115 [cs.CL]. URL: https://arxiv.org/abs/2412.15115.

[45] Alec Radford and Karthik Narasimhan. "Improving Language Understanding by Generative Pre-Training". In: 2018. URL: https://api.semanticscholar.org/CorpusID:49313245.

[46] Colin Raffel et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: (2023). arXiv: 1910.10683 [cs.LG]. URL: https://arxiv.org/abs/1910.10683.

[47] John W. Ratcliff and David E. Metzener. "Pattern Matching: The Gestalt Approach". In: *Dr. Dobb's Journal* 13.7 (1988), p. 46.

[48] Nils Reimers and Iryna Gurevych. "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2020. URL: https://arxiv.org/abs/2004.09813.

[49] Rebecca Salganik, Fernando Díaz, and Golnoosh Farnadi. "Fairness Through Domain Awareness: Mitigating Popularity Bias for Music Discovery". In: Mar. 2024, pp. 351–368. ISBN: 978-3-031-56065-1. DOI: 10.1007/978-3-031-56066-8_27.

[50] Ben Schafer et al. "Collaborative Filtering Recommender Systems". In: Jan. 2007.

[51] Wei Sun and Revathi Sundarasekar. "Research on pattern recognition of different music types in the context of AI with the help of multimedia information processing". In: *ACM Transactions on Asian and Low-Resource Language Information Processing* (Feb. 2023). DOI: 10.1145/3523284.

[52] TIME. *Big Tech's Race to Deploy RAG-Powered Assistants*. https://time.com/7012883/patrick-lewis/. 2024.

[53] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971.

[54] Robin Ungruh et al. "Putting Popularity Bias Mitigation to the Test: A User-Centric Evaluation in Music Recommenders". In: *Proceedings of the 18th ACM Conference on Recommender Systems*. RecSys '24. Bari, Italy: Association for Computing Machinery, 2024, 169–178. ISBN: 9798400705052. DOI: 10.1145/3640457.3688102. URL: https://doi.org/10.1145/3640457.3688102.

[55] Tong Xiao and Jingbo Zhu. *Foundations of Large Language Models*. 2025. arXiv: 2501.09223 [cs.CL]. URL: https://arxiv.org/abs/2501.09223.

[56] Yi-hsuan Yang and Jen-Yu Liu. "Quantitative Study of Music Listening Behavior in a Social and Affective Context". In: *Multimedia, IEEE Transactions on* 15 (Oct. 2013), pp. 1304–1315. DOI: 10.1109/TMM.2013.2265078.

[57] Michihiro Yasunaga et al. *Retrieval-Augmented Multimodal Language Modeling*. 2023. arXiv: 2211.12561 [cs.CV]. URL: https://arxiv.org/abs/2211.12561.

[58] Huixue Zhou et al. “The Efficiency vs. Accuracy Trade-off: Optimizing RAG-Enhanced LLM Recommender Systems Using Multi-Head Early Exit”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Wanxiang Che et al. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 26443–26458. ISBN: 979-8-89176-251-0. DOI: 10.18653/v1/2025.acl-long.1283. URL: https://aclanthology.org/2025.acl-long.1283/.